

Classification and Prediction

⊗ Classification: Classification is the process of finding or discovering a model or function which helps to predict class label for a given data. For example: we can build a classification model to categorize bank loan applications as either safe or risky. We can also construct a classification model to identify digits.

⊗ Prediction: Prediction is the process of finding a model or function which is used to predict continuous real-valued output for a given data. For example: we can build a regression model to predict the expenditures of a potential customers on computer equipment given their income and occupation. We can also build a prediction model to predict stock price given historical trading data.

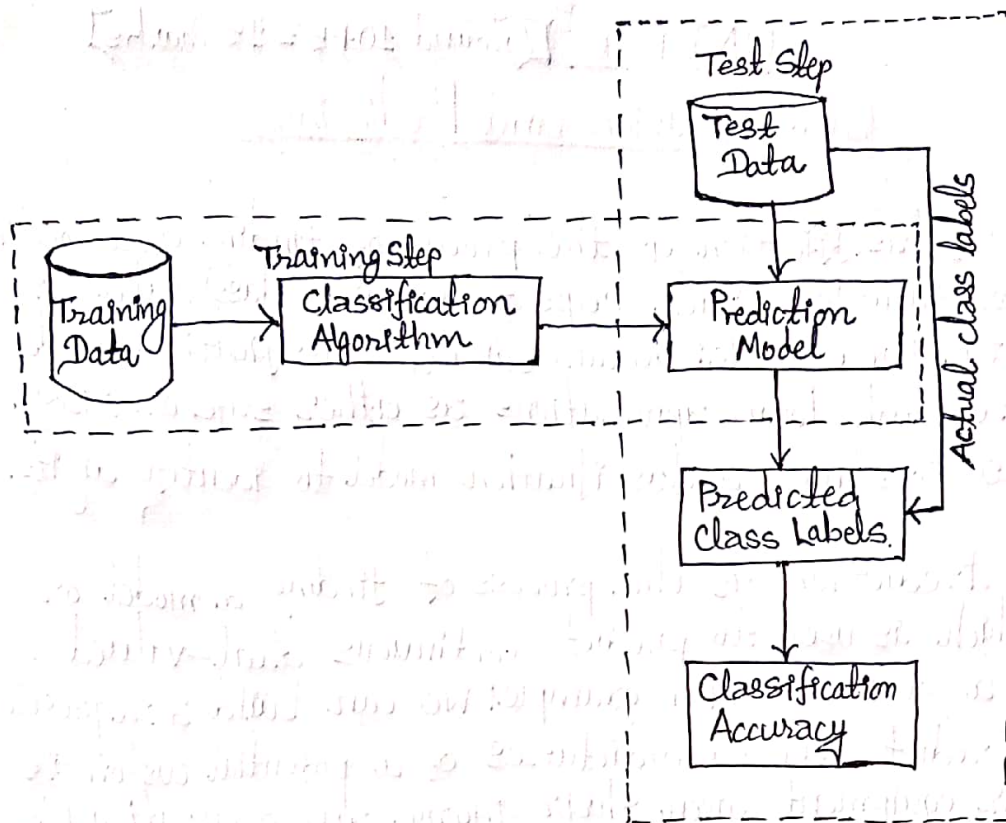
⊗ Learning and Testing of Classification:

The classification process works in following two steps: Learning Step and Testing Step.

Learning Step: This step is also called training step. In this step the learning algorithms build a model on the basis of relationship between input and output in the training dataset. This dataset contains input attributes along with class label for every input tuple.

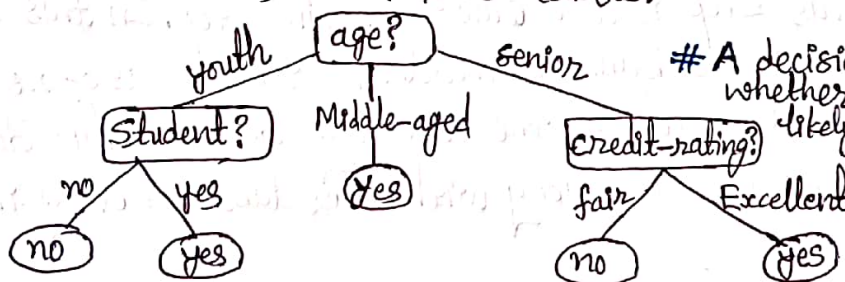
Testing Step: In this step, the model is used for prediction. Here the test dataset is used to estimate the accuracy of the model. This dataset contains values of input attributes along with class label of the output attribute.

Accuracy of the model is computed by looking at predicted class labels and actual class labels of test dataset. The model can be applied to the new data tuples if the accuracy is considered acceptable.



Classification by decision tree induction:

Decision tree induction is the learning of decision trees from class labeled training tuples. Decision tree is a flowchart-like tree structure where internal nodes (non leaf node) denotes a test on an attribute, branches represent outcomes of tests, and leaf nodes (terminal nodes) hold class labels.



A decision tree indicating whether a customer is likely to purchase a computer

Class-label Yes: The customer is likely to buy a computer.

Class-label No: The customer is unlikely to buy a computer.

In order to make prediction for a tuple, the attributes of a tuple are tested against the decision tree. A path is traced from the root to a leaf node which determines the predicted class for that tuple.

⊗ Why decision trees classifiers are so popular?

- The construction of a decision tree does not require any domain knowledge or parameter setting.
- They can handle high dimensional data.
- Intuitive representation that is easily understood by humans.
- Learning and classification are simple and fast.
- They have a good accuracy.

Algorithm for Constructing Decision Trees:

Constructing a Decision tree uses greedy algorithm. Tree is constructed in a top-down divide-and-conquer manner.

1. At start, all the training tuples are at the root.
2. Tuples are partitioned recursively based on selected attributes.
3. If all samples for a given node belong to the same class
→ label the class
4. Else if there are no remaining attributes for further partitioning
→ Majority voting is employed for assigning class label to the leaf.
5. Else
→ Go to step 2.

There are many variations of decision-tree algorithms. Some of them are: ID3 (Iterative Dichotomiser 3), C4.5 (successor of ID3), CART (Classification and Regression Tree) etc.

There are different attribute selection measures used by decision tree classifiers. Some of them are: Information Gain, Gain Ratio, Gini Index etc.

⊗ ID3 Decision Tree: [Imp]

ID3 stands for Iterative Dichotomiser 3. It uses top-down greedy approach to build decision tree model. This algorithm computes information gain for each attribute and then selects the attribute with the highest information gain. Information gain measures reduction in entropy after data transformation. It is calculated by comparing entropy of the dataset before and after transformation.

⇒ Entropy is the measure of homogeneity of the sample. Entropy or expected information of dataset D is calculated by using equation 1 as follows:

$$E(D) = - \sum_{i=1}^m p_i \log_2 p_i \quad \text{--- (1)}$$

where p_i is the probability of the tuple in D belonging to class C_i and is estimated using Equation 2 as follows:

$$p_i = \frac{|C_{i,D}|}{|D|} \quad \text{--- (2)}$$

where $|C_{i,D}|$ is the number of tuples in D belonging to class C_i and $|D|$ is the number of tuples in D .

⇒ Suppose we have to partition the tuples in D on some attribute A having v distinct values. The attribute A can be used to split D into v partitions $\{D_1, D_2, \dots, D_v\}$. Now, the total entropy of data partitions while partitioning D around attribute A is calculated using Equation 3 as follows:

$$E_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times E(D_j) \quad \text{--- (3)}$$

⇒ Finally, the information gain achieved after partitioning D on attribute A is calculated using Equation 4 as follows:-

$$IG(A) = E(D) - E_A(D) \quad \text{--- (4)}$$

Example: Construct decision tree from given data using information gain.
 Predict class level of the tuple: $X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair})$.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle-aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle-aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle-aged	medium	no	excellent	yes
13	middle-aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Solution:

Expected information needed to classify a tuple in D is:

$$E(D) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) = 0.94$$

Expected information needed to classify a tuple in D if the tuples are partitioned according to age is:

$$E_{\text{age}}(D) = \frac{5}{14} \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) + \frac{4}{14} \left(-\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} \right) + \frac{5}{14} \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right) = 0.694$$

$$\text{Gain}(\text{Age}) = E(D) - E_{\text{age}}(D) = 0.94 - 0.694 = 0.246$$

Similarly we will calculate for other partitioned tuple for their Expected information and Gain for remaining ones (income, student, credit_rating).

माफ़ table मा total 14 Data है (D) जसमें 9 class हा 9 Data yes है 5 Data no. So $\frac{9}{14}$ भाकी probability र $\frac{5}{14}$ भाकी probability formula का calculation करेंगे।
 $m=2$ since we have 2 class.

अब हमें age को लागि देखें हैं। Total 3 type की age है 80, m=3. youth की probability $\frac{5}{14}$ है 80, बाकि $\frac{5}{14}$ मात्र की कुल माफ़ $E(D)$ जसमें youth yes भाकी 5 मध्ये 2 है 80, $\frac{2}{5}$ probability / लयसे गरि youth no भाकी 3 मध्ये 3 है 80, $\frac{3}{5}$. Similarly for middle-aged and senior. Note: Partition tuple लई yes, no होवी सही class सँग देखेको, यो $E(D)$ को formula use भाकी

formula of eqn(4)

exam मा सबै गरे देखेउन नो दोटा लगाउन process same भए calculation skip गरेको उनको लागि

Then we will get,

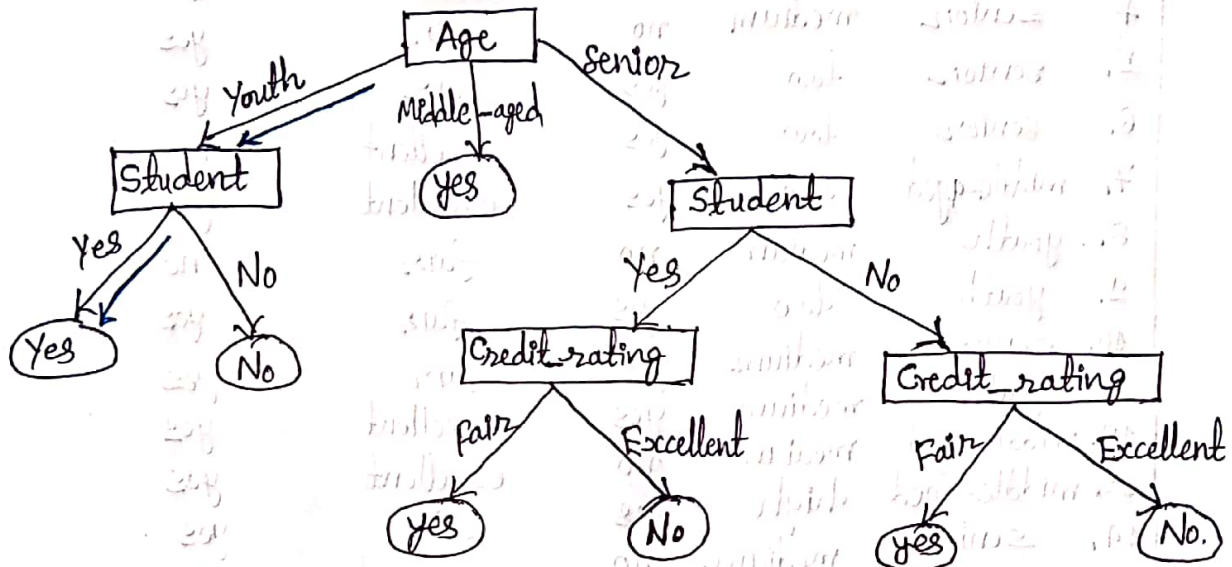
$$\text{Gain}(\text{student}) = 0.151$$

$$\text{Gain}(\text{Income}) = 0.029$$

$$\text{Gain}(\text{credit_rating}) = 0.048$$

जैसे तो Gain बढे
होने को top में आयेगा
जैसे कम हो
max to min गेने
bottom में
को आयेगा

Thus we partition the tree in the order of attributes: age, student, credit_rating, and income respectively. Thus we get following decision tree.



The additional arrow above ($\rightarrow$) is for the ~~the~~ Given tuple:
 $X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair})$.
 This from above decision tree, the predicted class for the given data is: Buy_computer = Yes.

* Bayesian Classification: [Impl]

Bayesian classification is based on Bayes theorem. It is also called Naïve Bayes Classification or Naïve Bayesian Classification.

Bayes Theorem is given by:

$$P(H|X) = \frac{P(X|H) P(H)}{P(X)}$$

Bayes theorem is useful in that it provides a way of calculating the posterior probability, $P(H|X)$, from $P(H)$, $P(X|H)$, and $P(X)$. Here $P(X)$ and $P(H)$ are prior probability.

Let D be a database and $C_1, C_2, \dots, C_m$ are m classes. Now above Bayes rule can be written as:

$$P(C_i|X) = \frac{P(X|C_i) P(C_i)}{P(X)}$$

Given a tuple, X , the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X . Thus we need to maximize $P(C_i|X)$. As $P(X)$ is constant for all classes, only $P(X|C_i) P(C_i)$ need to be maximized.

Let X is the set of attributes $\{x_1, x_2, x_3, \dots, x_n\}$ where attributes are independent of one another. Now the Probability $P(X|C_i)$ is given by the equation given below:

$$P(X|C_i) = \prod_{k=1}^n P(x_k|C_i) = P(x_1|C_i) \times P(x_2|C_i) \times \dots \times P(x_n|C_i).$$

Example: Predict class level of the tuple: $X = (\text{age}=\text{youth}, \text{income}=\text{medium}, \text{student}=\text{yes}, \text{credit_rating}=\text{fair})$ using Bayesian classification.

RID	Age	Income	Student	credit_rating	Class: buys computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle-aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle-aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle-aged	medium	no	excellent	yes
13	middle-aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Solution:

Prior probability of each class can be computed based on the training tuples:

$$P(\text{buys_computer}=\text{yes}) = 9/14 = 0.643$$

$$P(\text{buys_computer}=\text{no}) = 5/14 = 0.357$$

Compute the conditional probabilities:

$$P(\text{age}=\text{youth} | \text{buys_computer}=\text{yes}) = 2/9 = 0.222$$

$$P(\text{age}=\text{youth} | \text{buys_computer}=\text{no}) = 3/5 = 0.6$$

$$P(\text{income}=\text{medium} | \text{buys_computer}=\text{yes}) = 4/9 = 0.444$$

$$P(\text{income}=\text{medium} | \text{buys_computer}=\text{no}) = 2/5 = 0.4$$

$$P(\text{student}=\text{yes} | \text{buys_computer}=\text{yes}) = 6/9 = 0.667$$

$$P(\text{student}=\text{yes} | \text{buys_computer}=\text{no}) = 1/5 = 0.2$$

$$P(\text{credit_rating}=\text{fair} | \text{buys_computer}=\text{yes}) = 6/9 = 0.667$$

$$P(\text{credit_rating}=\text{fair} | \text{buys_computer}=\text{no}) = 2/5 = 0.4$$

Using the above probabilities, we obtain:

$$\begin{aligned} P(X | \text{buys_computer}=\text{yes}) &= P(\text{age}=\text{youth} | \text{buys_computer}=\text{yes}) \times \\ &P(\text{income}=\text{medium} | \text{buys_computer}=\text{yes}) \times P(\text{student}=\text{yes} | \text{buys_computer}=\text{yes}) \times \\ &P(\text{credit_rating}=\text{fair} | \text{buys_computer}=\text{yes}) \\ &= 0.222 \times 0.444 \times 0.667 \times 0.667 \\ &= 0.044 \end{aligned}$$

$$\begin{aligned} P(X | \text{buys_computer}=\text{no}) &= P(\text{age}=\text{youth} | \text{buys_computer}=\text{no}) \times P(\text{income}=\text{medium} | \text{buys_computer}=\text{no}) \times \\ &P(\text{student}=\text{yes} | \text{buys_computer}=\text{no}) \times \\ &P(\text{credit_rating}=\text{fair} | \text{buys_computer}=\text{no}) \\ &= 0.6 \times 0.4 \times 0.2 \times 0.4 \\ &= 0.019 \end{aligned}$$

$$\begin{aligned} \text{Now, } P(\text{buys_computer}=\text{yes} | X) &= P(X | \text{buys_computer}=\text{yes}) \times P(\text{buys_computer}=\text{yes}) \\ &= 0.044 \times 0.643 \\ &= 0.028 \end{aligned}$$

$$\begin{aligned} P(\text{buys_computer}=\text{no} | X) &= P(X | \text{buys_computer}=\text{no}) \times P(\text{buys_computer}=\text{no}) \\ &= 0.019 \times 0.357 \\ &= 0.007 \end{aligned}$$

Since, $P(\text{buys_computer}=\text{yes} | X) > P(\text{buys_computer}=\text{no} | X)$,
Prediction for X is: $\text{buys_computer} = \text{yes}$.

question मा दिएको हामी
लागि मात्र calculate गर्ने

question मा दिएको
सबै रोक रोक
buys: yes र no को
माफत

* Laplace Smoothing:

- If probability value is zero for some $P(x_k | C_i)$, it results to zero probability for $P(x | C_i)$.
- Zero probability cancels the effects of all the other probabilities involved in the product.
- Laplace Smoothing is the technique to solve this problem. It adds 1 to each count. The main philosophy behind the concept is that the database is normally large and adding 1 to the count makes negligible difference in probability.

* Classification By Backpropagation: [Imp]

A popular method for the training of multilayer perceptrons is the backpropagation algorithm. The training proceeds in two phases:

Forward Phase: In this phase, the synaptic weights of the network are fixed and the input signal is propagated through the network, layer by layer, until it reaches the output. Thus in this phase, changes are confined to the activation potentials and outputs of the neurons in the network.

Backward Phase: In this phase, an error signal is produced by comparing the output of the network with a desired response. The resulting error signal is propagated in backward direction through the network, again layer by layer. In this second phase, successive adjustments are made to the synaptic weights of the network.

Algorithm:

1. Initialize all weights and biases in network.
 2. While terminating condition is not satisfied
 3. for each training tuple X in D
 4. for each input layer unit j : $O_j = I_j$; // Output is same as input value
 5. for each hidden or output layer unit j , compute the net input of unit j with respect to the previous layer, $I_j = \sum_i W_{ij} O_i + Q_j$
- Compute the output of each unit j , $O_j = \frac{1}{1 + e^{-I_j}}$

→ I_j is input O_j is output,
 j is neuron

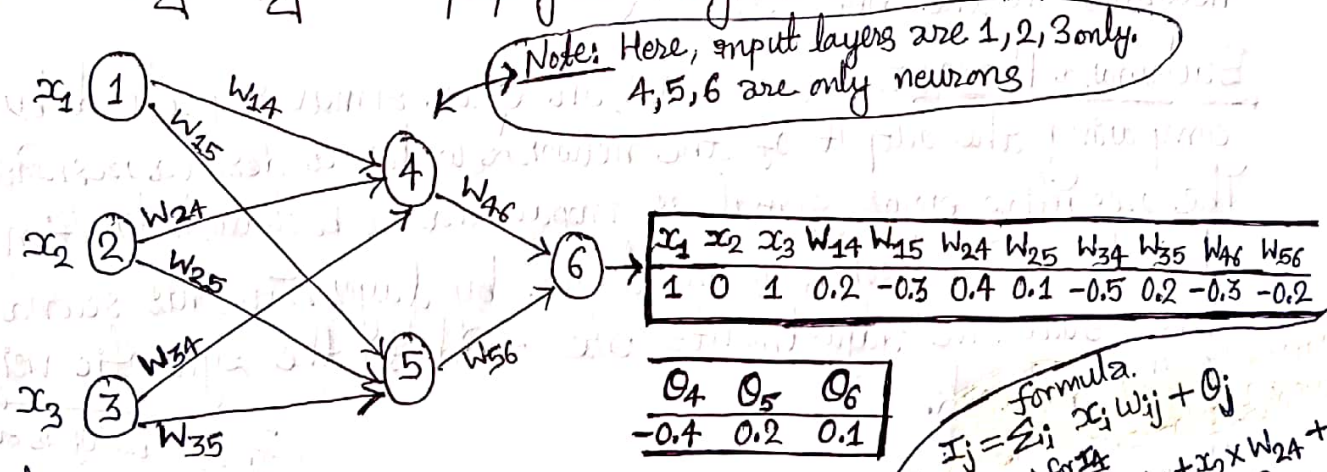
read as
theta which
is bias

forward
phase completed
here, now
backward
starts

← backward phase starts

6. for each unit j in the output layer
 $Err_j = O_j(1 - O_j)(T_j - O_j)$ ← Err_j is error, T_j is target output & O_j is calculated output
7. for each unit j in the hidden layers, from the last to the first hidden layer
 $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$
8. for each weight w_{ij} in network
 $\Delta w_{ij} = (l) Err_j O_i$, where l is learning rate, w_{ij} is weight increment
 $w_{ij} = w_{ij} + \Delta w_{ij}$ // weight update
9. for each bias O_j in network
 $\Delta O_j = (l) Err_j$ // bias increment
 $O_j = O_j + \Delta O_j$ // bias update

Example: Consider a multilayer feed-forward neural network given below. Let the learning rate be 0.9. The initial weight and bias values of the network are given in table, along with the first training tuple, $X = (1, 0, 1)$, whose target output is 1. Show weight updates by using back-propagation algorithm.



Solution:

Net input and output calculation:

Unit j	Net input, I_j	Output, O_j
4	$0.2 + 0 - 0.5 - 0.4 = -0.7$	$1/(1 + e^{0.7}) = 0.332$
5	$-0.3 + 0 + 0.2 + 0.2 = 0.1$	$1/(1 + e^{-0.1}) = 0.525$
6	$(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$	$1/(1 + e^{0.105}) = 0.474$

j denotes neurons

at least one calculator
~~calculator~~
 try it
 Exponential calculate जिन सिकिने

formula:
 $I_j = \sum_i x_i w_{ij} + O_j$
 Eg. let for I_4
 $I_4 = x_1 \times w_{14} + x_2 \times w_{24} + x_3 \times w_{34} + O_4$
 Now, placing values and calculating we get -0.7

formula:
 $O_j = \frac{1}{1 + e^{-I_j}}$
 Eg. I_4
 $O_4 = \frac{1}{1 + e^{-(-0.7)}}$

Errors at each node

For Output layer 6
 formula $Err_j = O_j (1 - O_j) (T_j - O_j)$
 for 6: $Err_6 = O_6 (1 - O_6) (T_6 - O_6)$
 now we put values and calculate as in the table. Value of T_j is 1 which is given in question

Unit j	Err _j
6	$(0.474) (1 - 0.474) (1 - 0.474) = 0.1311$
5	$(0.525) (1 - 0.525) (0.1311) (-0.2) = -0.0065$
4	$(0.332) (1 - 0.332) (0.1311) (-0.3) = -0.0087$

For hidden layer 5, 4 we use formula: $Err_j = O_j (1 - O_j) \sum_k Err_k W_{jk}$
 let for 5: $Err_5 = O_5 (1 - O_5) \sum_k Err_k W_{5k}$, Here k is no. of connected neurons, here we have only one, if two or more than two include by adding $\sum_k$
 now we put values and calculate as done in above table. Similarly for neuron 4.

Calculate weight and bias update

formula for weight update
 $w_{ij} = w_{ij} + (l) Err_j O_i$
 Note: l is learning rate given 0.9 in question

Weight or bias	New value
w_{46}	$-0.3 + (0.9) (0.1311) (0.332) = -0.261$
w_{56}	$-0.2 + (0.9) (0.1311) (0.525) = -0.138$
w_{14}	$0.2 + (0.9) (-0.0087) (1) = 0.192$
w_{15}	$-0.3 + (0.9) (-0.0065) (1) = -0.306$
w_{24}	$0.4 + (0.9) (-0.0087) (0) = 0.4$
w_{25}	$0.1 + (0.9) (-0.0065) (0) = 0.1$
w_{34}	$-0.5 + (0.9) (-0.0087) (1) = -0.508$
w_{35}	$0.2 + (0.9) (-0.0065) (1) = 0.194$
O_6	$0.1 + (0.9) (0.1311) = 0.218$
O_5	$0.2 + (0.9) (-0.0065) = 0.194$
O_4	$-0.4 + (0.9) (-0.0087) = -0.408$

Eg: $w_{34} = w_{34} + (l) * Err_4 * x_3$
 (Since $O_j = T_j$ in alg, $\therefore O_j = x_j$)
 $= -0.5 + 0.9 * (-0.0087) * 1$
 $= -0.508$
 similarly for others

formula for bias update
 $O_j = O_j + (l) Err_j$
 Eg: $O_6 = O_6 + 0.9 * Err_6$
 $= 0.1 + 0.9 * 0.131$
 $= 0.218$
 similarly for O_5, O_4

⊗ Rule Based Classification:

A rule-based classifier uses a set of If-Then rules for classification. An If-Then rule is an expression of the form

IF condition THEN conclusion.

An example is rule R1,

R1: If age = youth AND student = 'yes' Then buys_computer = 'yes'

If part consists of one or more attribute tests that are logically ANDed. Then part contains a class prediction. If the condition in If part is satisfied then the then part is satisfied.

The factors that measure quality of rule R are: coverage and accuracy.
We can define the coverage and accuracy of R as below:

$$\text{coverage}(R) = \frac{n_{\text{covers}}}{|D|}$$

where;

D denotes the total no. of tuples,

n_{covers} denotes no. of tuples satisfied by Rule(R).

& n_{correct} denotes no. of tuples correctly classified by Rule(R).

$$\text{accuracy}(R) = \frac{n_{\text{correct}}}{n_{\text{covers}}}$$

Example: Find coverage and accuracy of the rule

R1: If age=youth AND student='yes' Then buys_computer='yes'.

Table: Same table that we used in ID3 and Bayesian classification, in example.

same table पटक पटक repeatedly draw गर्ने, सम लोगन so, यदि पुरानो table हो question मा थप नभई सोच्ने हैन यदि table हैन,

Solution:

The rule covers 2 of the 14 tuples.

i.e, $n_{\text{cover}} = 2$ & $|D| = 14$

i.e, age=youth भएको र student=yes भन्ने 14 मध्ये 2 वटा टुपल table मा OR question को If part satisfy गर्ने no. in table

$$\text{Therefore, coverage}(R1) = \frac{n_{\text{covers}}}{|D|} = \frac{2}{14} = 14.28\%$$

No. of tuples correctly classified by rule is 2

i.e, $n_{\text{correct}} = 2$

$$\text{Therefore, accuracy}(R1) = \frac{n_{\text{correct}}}{n_{\text{covers}}}$$

$$= \frac{2}{2}$$

$$= 100\%$$

साथिको फलतः लोगको n_{cover} मध्ये $\text{buys_computer} = \text{yes}$ कतिमा छ त्यो value, यसमा दुई मध्ये दुईवटैमा छ so, 2.
OR Question को If र then दुवै part कति tuple लाई satisfy गर्दछ त्यो no.

* Support Vector Machine:

Support Vector Machine or SVM is one of the most popular Supervised learning algorithms, which is used for Classification as well as Regression problems. However, it is mainly used for Classification problems in Machine Learning.

A support vector machine takes input data points and outputs the hyperplane. Hyperplane is the decision boundary. Decision boundary line separates classification of classes. Compared to newer algorithms like neural networks this have higher speed and better performance with a limited number of samples (in the thousands).

Among many possible hyperplanes, SVM algorithm selects optimal hyperplane by choosing hyperplane with largest margin. Such hyperplane is called maximum marginal hyperplane (MMH). SVM can be of two types: Linear SVM and Non-Linear SVM. SVM that is used to classify linearly separable data points is called linear SVM whereas the SVM that is used to classify non-linearly separable data points is called non-linear SVM.

Example: Consider the following data points:

Positively Labelled Data Points: $(3, 1), (3, -1), (6, 1), (6, -1)$

Negatively Labelled Data Points: $(1, 0), (0, 1), (0, -1), (-1, 0)$

Determine the equation of hyperplane that divides the above data points into two classes:

Solution:

Support vectors are:

$$S_1 = (1, 0), S_2 = (3, 1), S_3 = (3, -1)$$

Augment support vectors with $b=1$

$$S_1 = (1, 0, 1), S_2 = (3, 1, 1), S_3 = (3, -1, 1)$$

Since there are three support vectors, we need to calculate three variables. Thus, three linear equations can be written as:

$$\alpha S_1 \cdot S_1 + \beta S_2 \cdot S_1 + \lambda S_3 \cdot S_1 = -1$$

$$\alpha S_1 \cdot S_2 + \beta S_2 \cdot S_2 + \lambda S_3 \cdot S_2 = 1$$

$$\alpha S_1 \cdot S_3 + \beta S_2 \cdot S_3 + \lambda S_3 \cdot S_3 = 1$$

After simplifying above equations, we get:

$$2\alpha + 4\beta + 4\lambda = -1$$

$$4\alpha + 11\beta + 9\lambda = 1$$

$$4\alpha + 9\beta + 11\lambda = 1$$

Solving these equations, we get

$$\alpha = -3.5, \beta = 0.75, \lambda = 0.75$$

Now, we can compute weight vector of hyperplane as below:

$$W = \alpha S_1 + \beta S_2 + \lambda S_3 = (1, 0, -2)$$

Thus, $W = (1, 0)$ and $b = (-2)$

The line is vertical line parallel to y-axis. Hence, equation of hyperplane that divides data points is $x - 2 = 0$.

⊗ Performance Measures of Classification: [Imp]

A Confusion matrix is an $N \times N$ matrix used for evaluating the performance of a classification model, where N is the number of target classes. For a binary classification problem, we would have a 2×2 matrix as shown below:

		Actual Classes	
		Positive	Negative
Predicted Classes	Positive	TP	FP
	Negative	FN	TN

True Positive (TP): It represents correctly classified positive classes. Both actual and predicted classes are positive here.

False Positive (FP): It represents incorrectly classified positive classes. These are the positive classes predicted by the model that were actually negative. This is called Type I error.

True Negative (TN): It represents correctly classified negative classes. Both actual and predicted classes are negative here.

False Negative (FN): It represents incorrectly classified negative classes. These are negative classes predicted by the model that were actually positive. This is called Type II error.

Four widely used performance measures used for evaluating classification models are: Accuracy, Recall, Precision, F1-score.

Accuracy: It is the percentage of correct predictions made by the model and is given as below:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} = \frac{TP+TN}{\text{Total number of instances}}$$

Precision: It is the percentage of predicted positives that are actually positive and is given by:

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{TP}{\text{Total number of Predicted Positives}}$$

Recall: It is the percentage of actual positives that are correctly classified by the model and is given as below:

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{TP}{\text{Total number of actual positives}}$$

F1-score: It is the harmonic mean of recall and precision. It becomes high only when both precision and recall are high. This score is given by:

$$F1 = \frac{2 \times \text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}$$

⊗ Cross-Validation:

Cross-validation is a re-sampling procedure used to evaluate machine learning models on a limited data sample. k-fold Cross-Validation is widely used for cross validation. If we choose $k=10$, it becomes 10-fold cross validation.

In this approach, the original sample is randomly partitioned into k equal sized subsamples. Out of the k subsamples, a single subsample is used as the testing set, and the remaining $k-1$ subsamples are used as a training set. The cross-validation process is then repeated k -times, with each of the k subsamples used exactly once as the testing data. The k results can then be averaged to produce a single estimation. The advantage of this method is that all observations are used for both training and testing, and each observation is used for testing exactly once.

Example: Let we have a data sample with 6 observations:

$\{(1,3), (2,5), (3,7), (4,9), (5,11), (6,13)\}$

Let $k=3$. That means we will shuffle the data and then split the data into 3 groups.

Fold 1 = $\{(1,3), (5,11)\}$

Fold 2 = $\{(6,13), (3,7)\}$

Fold 3 = $\{(2,5), (4,9)\}$

Three models are trained and evaluated with each fold given a chance to be the held out test set. For example:

Model 1: Trained on Fold 1 + Fold 2, Tested on Fold 3.

Model 2: Trained on Fold 2 + Fold 3, Tested on Fold 1.

Model 3: Trained on Fold 1 + Fold 3, Tested on Fold 2.

⊗ Overfitting and Underfitting: [Imp]

Overfitting is the result of using an excessively complicated model. It happens when the model runs the details and noise of the training data to the extent that it negatively impacts the performance of the model on new data. This means that the model learns noise or random fluctuations in the training data as concepts. This negatively impacts the model's ability to generalize because these concepts do not apply to new data.

Underfitting is the result of using an excessively simple model or using very few training samples. In such situations, a machine learning algorithm cannot capture the underlying trend of the data. An underfitted machine learning model is not a suitable model for making predictions because it has poor performance on the training data too.

* McNemar's Test: (Comparing ^{OR} two classifiers) [Imp]

In terms of comparing two binary classification algorithms, McNemar's test is the test commenting on whether the two models disagree in the same way (or not). It is not commenting on whether one model is more or less accurate or error prone than another. In McNemar's test, we begin by creating a contingency table like the one below to compare the performance of model 1 and model 2.

a	1148	66	b
c	43	166	d

In this table, the regions are:

a → both models are correct

b → Model 1 is correct, and model 2 is wrong.

c → Model 2 is correct, and model 1 is wrong.

d → both models are wrong.

We can then compute the chi-square statistic as:

$$\chi^2 = \frac{(b-c)^2}{(b+c)}$$

Additionally, we need to obtain a p-value from the Chi-Square test. p-value is the area under the Chi-Square density. If the p-value is greater than 0.05, it can be concluded that there isn't a significant difference between false negatives and false positives of two classification algorithms. And, if the p-value is less than or equal to 0.05, it means there is a significant difference.

⊗ Classification Issues: [Imp]

The major issues in preparing the data for Classification and Prediction involves following activities.

Data Cleaning: It involves removing the noise and treatment of missing values.

Relevance Analysis: Database may also have the irrelevant attributes. Correlation analysis is used to know whether any two given attributes are related.

Data Normalization: Data is transformed using normalization.

Normalization involves scaling all values for given attribute in order to make them fall within a small specified range.

$$\frac{x - d}{(c - d)} = r$$